Mobile UI/UX Specification Report: Labella

This report provides a detailed technical blueprint for a Flutter developer to replicate the **Labella** web design system, styling, and components into a high-end mobile application.

---

1. Visual Identity & Design Philosophy

Labella's aesthetic is **Luxury Minimalist** with a "Glassmorphism" theme. It focuses on soft transitions, premium typography, and a refined color palette that feels both modern and elegant.

Core Principles

- **Glassmorphism**: Heavy use of backdrop blurs and semi-transparent layers.
- **Premium Motion**: Subtle, fluid animations (Fade-ups, scale effects).
- **Responsive Hierarchy**: Clear distinction between display (headings) and body text.

---

2. Design System (Tokens)

A. Color Palette (HSL to Hex/ARGB)

Labella supports both **Light** and **Dark** modes. In Flutter, use `ThemeData` to manage these.

Token	Light Mode (Hex)	Dark Mode (Hex)	Purpose
Background	`#FCF9F2` (Cream)	`#1D1418` (Deep Plum)	Main app background
Foreground	`#2E2E2E`	`#FDFBF8`	Primary text
Primary	`#E5A1BA` (Rose)	`#E5A1BA`	Key actions / Highlights
Secondary	`#7A405D` (Plum)	`#A36684`	Dark accents / Buttons
Accent	`#D4A373` (Gold)	`#D4A373`	Badges / Alerts
Muted	`#F5EBE0`	`#3A2A32`	Background elements
Card	`#FBF7F0`	`#2D2026`	Card backgrounds
Border	`#F2D5E0`	`#3F323A`	Dividers / Outlines

B. Typography

Replicate using the `google_fonts` package.

1. **Display Font**: `Playfair Display` (Serif)

    - *Usage*: Headings (h1, h2, h3).
    - *Feel*: Elegant, authoritative.
2. **Body Font**: `Poppins` (Sans-serif)
    - *Usage*: Paragraphs, buttons, inputs.
    - *Feel*: Modern, clean.
3. **Script Font**: `Great Vibes` (Cursive)
    - *Usage*: Branding (Logo), specific decorative accents.

C. Effects & Spacing

- **Border Radius**: Default is `16.0` (`1rem`). Use `BorderRadius.circular(16)`.
- **Gradients**:
    - `Hero`: `LinearGradient(begin: Alignment.topLeft, end: Alignment.bottomRight, colors: [Soft, Cream, Primary.withOpacity(0.45)])`
    - `Plum`: `LinearGradient(colors: [Secondary, Primary])`
- **Shadows**:
    - `Warm`: `BoxShadow(color: Secondary.withOpacity(0.38), blurRadius: 55, offset: Offset(0, 18))`
    - `Glow`: `BoxShadow(color: Primary.withOpacity(0.72), blurRadius: 48, offset: Offset(0, 16))`

---

3. Component Architecture

A. Global Layout (`Layout.tsx` Mapping)

- **Header (Sticky)**: Use `SliverAppBar` with `backdropFilter` (Blur).
- **Navigation**:
    - **Desktop/Tablet**: Side Rail or Centered Nav.
    - **Mobile**: `BottomNavigationBar` with icons: Home, Shop, Cart, Wishlist, Profile.
- **Glass Panel**: A custom container widget.

```
Container(
  decoration: BoxDecoration(
    color: Colors.white.withOpacity(0.1),
    borderRadius: BorderRadius.circular(16),
    border: Border.all(color: Colors.white.withOpacity(0.05)),
  ),
  child: BackdropFilter(
    filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),
    child: child,
  ),
)
```

B. Product Card (`ProductCard.tsx` Mapping)

- **Structure**:
  1. **Image**: `CachedNetworkImage` with a `shimmer` placeholder. `BoxFit.contain` inside an aspect ratio of `4:5`.
  2. **Badges**: Absolute positioned (Stack) `Sale` (Gold) or `Out of Stock` (Plum) labels.
  3. **Actions**: Top-right `Heart` button (Wishlist). Bottom-right `ShoppingBag` button.
  4. **Content**: Title (Max 2 lines), Rating stars (Accent color), Price (Secondary color, bold).

C. Buttons

- **Hero Button**: Gradient background (Plum/Rose), white text, high elevation.
- **Glass Button**: Semi-transparent background, subtle border, blur effect.
- **Icon Button**: Rounded-xl (12px), background variations based on state.

---

4. Interaction & Motion UX

A. Scroll Animations

The website uses `ScrollReveal` (Fade-up). In Flutter, use the `flutter_animate` package:

- **Fade-Up**: `myWidget.animate().fade(duration: 500.ms).moveY(begin: 20, end: 0)`
- **Scale-Hover**: On product cards, use `MouseRegion` (for web/desktop) or `GestureDetector` to scale slightly on press.

B. Micro-interactions

- **Heart Toggle**: Scale and bounce animation when favoriting.
- **Cart Pulse**: When adding to cart, the cart icon should "jump" or pulse briefly.

- **Skeleton Loading**: Use a gradient shimmer for content that is loading.

---

5. Technical Implementation Checklist (Flutter)

Recommended Plugins

- `google_fonts`: For typography.
- `lucide_icons`: To match the website's iconography.
- `flutter_animate`: For all motion effects.
- `cached_network_image`: For optimized image handling.
- `flutter_riverpod` or `Bloc`: For state management (Cart, Wishlist, Auth).

Project Structure Suggestion

```
lib/
├── core/
│   ├── theme/          # ThemeData (Light/Dark)
│   ├── constants/      # App colors, gradients, dimensions
│   └── widgets/        # GlassPanel, SectionShell, Stars
├── features/
│   ├── store/          # ProductCard, CategoryGrid
│   ├── auth/           # Login/Register UI
│   └── cart/           # Cart components
└── main.dart           # App entry + Theme providers
```

---

6. RTL Support

Since the website supports **Arabic**, Ensure:

- `MaterialApp(localizationsDelegates: ..., supportedLocales: [Locale('en'), Locale('ar')])`
- Use `Directionality.of(context)` for specific mirrored layouts.
- Alignment should use `AlignmentDirectional` instead of `Alignment.left/right`.

Report generated for Labella Mobile Development Team.

---

7. Backend Integration & API Mapping

The mobile app must interface with the existing Node.js/Express backend. All requests should use the base URL provided in the environment configuration (typically `{{BASE_URL}}/api/v1/`).

A. Authentication & User

Action	Method	Endpoint	Data / Notes
Signup	`POST`	`/auth/signup`	name, email, password
Login	`POST`	`/auth/login`	email, password -> Returns JWT
Profile	`GET`	`/profile`	Requires JWT in Header
Update Profile	`PUT`	`/profile`	name, phone, etc.

B. Catalog & Discovery

Action	Method	Endpoint	Data / Notes
List Products	`GET`	`/products`	Supports `?keyword=`, `?category=`, `?sort=`, `?page=`
Product Detail	`GET`	`/products/:id`	Returns full product object + reviews
Categories	`GET`	`/categories`	List all main categories
Subcategories	`GET`	`/categories/:id/subcategories`	Filtered by parent category

C. Shopping & Personalization (Authenticated)

Action	Method	Endpoint	Data / Notes
Get Cart	`GET`	`/cart`	Retrieves current active cart
Add to Cart	`POST`	`/cart`	`{ productId: string }`
Update Quantity	`PUT`	`/cart/:id`	`{ quantity: number }`
Toggle Wishlist	`POST`	`/wishlist`	`{ productId: string }`
Get Wishlist	`GET`	`/wishlist`	Returns list of favorited products

D. Checkout & Fulfillment

Action	Method	Endpoint	Data / Notes
Get Addresses	`GET`	`/address`	Saved shipping addresses
Add Address	`POST`	`/address`	street, city, state, zip
Place Order	`POST`	`/order`	`{ address: addressId }`
Order History	`GET`	`/order`	List of past user orders

---

8. User Flow & Action Mapping

Flow 1: Discovery to Cart

1. **Action**: User opens app.
    - *API*: `GET /products?limit=10` (Trending) + `GET /categories`.
2. **Action**: User searches for "Lipstick".
    - *API*: `GET /products?keyword=Lipstick`.
3. **Action**: User clicks a product.
    - *API*: `GET /products/:id`.
4. **Action**: User clicks "Add to Cart".
    - *API*: `POST /cart` -> UI triggers "Cart Pulse" animation.

Flow 2: Checkout Process

1. **Action**: User opens Cart.
    - *API*: `GET /cart`.
2. **Action**: User clicks "Checkout".
    - *API*: `GET /address`.
3. **Action**: User selects or adds an address.
    - *API*: `POST /address` (if new).
4. **Action**: User confirms order.
    - *API*: `POST /order` -> UI shows "Order Confirmed" Lottie/Icon.

Flow 3: Personalization

1. **Action**: User taps "Heart" on a Product Card.
    - *API*: `POST /wishlist` -> UI toggles heart fill state.
2. **Action**: User views Profile.
    - *API*: `GET /profile` + `GET /order`.

---

9. Guidance for the Implementing AI

1. **Token Management**: Store the JWT securely (e.g., `flutter_secure_storage`). Include it in the `Authorization: Bearer <token>` header for all "Authenticated" endpoints.
2. **Error Handling**: The backend returns `ApiErrors` (e.g., 400 for validation). Map these to user-friendly Snackbars or inline validation.
3. **Optimistic UI**: For "Wishlist Toggle" and "Cart Quantity", update the UI locally *before* the API call completes to maintain a premium, snappy feel.
4. **Image Loading**: Use the `api.imgUrl()` logic from the web project (Base URL + path) to display product images correctly via `CachedNetworkImage`.

Report generated for Labella Mobile Development Team.